

Anno Accademico 2025/2026

Leonardo Donati

Appunti

Materia

Teoria e esercizi.



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Indice

1	Introduzione	3
1.1	Ripasso Algebra Lineare	3
1.2	Ripasso Statistica	3
1.3	Machine Learning	4
2	Apprendimento Supervisionato	5
2.1	Dati	5
2.2	Processo di Apprendimento Supervisionato	5
2.3	Apprendimento	6
2.3.1	Misure di valutazione	6
2.3.2	Modello di valutazione	6
2.4	Iperparametri	6
2.5	Altre Misure di Classificazione	7
3	Probabilistic Classifier and Linear Classification Models	8
3.1	Definizioni	8
3.1.1	Formula di Bayes	8
3.1.2	Maximum Likelihood	8
3.1.3	Maximum A-Posteriori	9
3.2	Classificatore Ottimo di Bayes	9
3.2.1	Stima della densità	9
3.2.2	Naive Density (Naive DE)	9
3.3	Linear Discriminant Analysis (LDA)	10
3.3.1	Quadratic Discriminant Analysis	10
3.4	Logistic Regression	11
3.4.1	Ottimizzazione dei Parametri	11
3.4.2	Discesa del Gradiente	12
4	Support Vector Machine	13
4.1	Classificazione Binaria	13
4.1.1	Problema quadratico SVM	14
4.1.2	Ottimizzazione Vincolata	14
4.1.3	Classificatore SVM	15
4.1.4	SVM non Lineare	15

Capitolo 1

Introduzione

1.1 Ripasso Algebra Lineare

DEFINITION Spazio vettoriale

Lo **spazio vettoriale** $\mathbb{R}^n$ è un insieme di elementi $x = [x_1, \dots, x_n]$ where each $x_i \in \mathbb{R}$. Gli elementi x sono chiamati **vettori** e soddisfano queste proprietà:

1. **Addizione:** Se $x \in \mathbb{R}^n$ e $y \in \mathbb{R}^n$, allora $x + y = [x_1 + y_1, \dots, x_n + y_n] \in \mathbb{R}^n$.
2. **Prodotto scalare:** Se $x \in \mathbb{R}^n$ e $a \in \mathbb{R}$ allora, $a \cdot x = [a \cdot x_1, \dots, a \cdot x_n]$.
3. **Prodotto interno:** Se $x \in \mathbb{R}^n$ e $y \in \mathbb{R}^n$ allora, $x \cdot y = \sum_{i=1}^n x_i \cdot y_i$.

DEFINITION Matrice

Una **matrice** $X \in \mathbb{R}^{n \times m}$ è un vettore rettangolare di elementi $x_{ij} \in \mathbb{R}$, $1 \leq i \leq n$, $1 \leq j \leq m$:

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1m} \\ x_{21} & x_{22} & \cdots & x_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nm} \end{pmatrix}$$

Deve supportare queste operazioni:

1. **Addizione:** Se $X \in \mathbb{R}^{n \times m}$, $Y \in \mathbb{R}^{n \times m}$ e $Z = X + Y$, allora $Z \in \mathbb{R}^{n \times m}$ e $Z_{ij} = X_{ij} + Y_{ij}$.
2. **Prodotto scalare:** Se $X \in \mathbb{R}^{n \times m}$, $a \in \mathbb{R}$ e $Z = a \cdot X$, allora $Z \in \mathbb{R}^{n \times m}$ e $Z_{ij} = a \cdot X_{ij}$.
3. **Prodotto fra matrici:** Se $X \in \mathbb{R}^{n \times m}$, $Y \in \mathbb{R}^{m \times n}$ e $Z = X \cdot Y$, allora $Z \in \mathbb{R}^{n \times n}$ e $Z_{ij} = \sum_{k=1}^m X_{ik} \cdot Y_{kj}$.

1.2 Ripasso Statistica

DEFINITION Distribuzione di Probabilità

Una distribuzione di probabilità P su uno spazio campionario Ω è un'applicazione dai sottoinsiemi di Ω ai numeri reali che soddisfa le seguenti condizioni:

- **Non-negatività:** $P(\alpha) \geq 0$, $\forall \alpha \subseteq \Omega$.
- **Normalizzazione:** $P(\Omega) = 1$.
- **Additività:** $\forall \alpha, \beta \subseteq \mathbb{R}$ che sono insiemi disgiunti, $P(\alpha \cup \beta) = P(\alpha) + P(\beta)$.

1.3 Machine Learning

DEFINITION Machine Learning

Il **machine learning** è un campo di studio che conferisce ai computer la capacità di apprendere senza essere esplicitamente programmati.

Esistono due tipi di approcci:

1. **Non parametrico:** l'apprendimento avviene memorizzando i dati di addestramento (memorizzazione).
2. **Parametrico:** l'apprendimento avviene utilizzando un algoritmo per adattare i parametri di un modello matematico o statistico sulla base di dati di addestramento. Per esempio:

$$f_{\theta}(x) = \sum_{d=1}^D \theta_d x_d$$

Capitolo 2

Apprendimento Supervisionato

2.1 Dati

Un insieme di record di dati $X \in X^k$, chiamati anche **esempi**, **istanze** o **casi** sono descritti da:

- k attributi (**Features**): $x = x_1, x_2, \dots, x_k$.
- Una **classe**: ogni esempio è etichettato con una classe target predefinita $c \in C$.

L'obiettivo è imparare un modello di classificazione dai dati che possono essere usati per predire la classe di nuovi casi.

2.2 Processo di Apprendimento Supervisionato

Il processo è diviso in due step:

1. Imparare un modello usando idati dell'allenamento.
2. Testare il modello con dati mai usati per vederne l'**accuratezza**.

$$\text{Accuratezza} = \frac{\text{Numero di classificazioni corrette}}{\text{Numero totali di casi}}$$

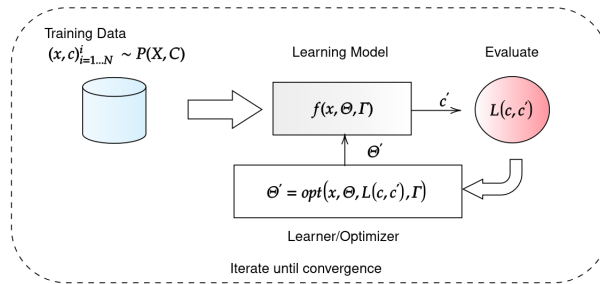


Figura 2.1: Training pipeline.

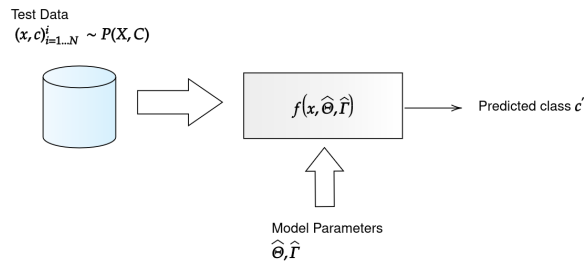


Figura 2.2: Inference pipeline.

2.3 Apprendimento

DEFINITION Apprendimento

Dati un dataset D , una classe compito C e una funzione di prestazione L , si dice che un sistema informatico apprende il compito C dai dati D se, dopo aver osservato D , la prestazione misurata tramite L migliora fino a raggiungere un livello superiore a quello di una scelta casuale.

In altre parole, i dati aiutano il sistema a migliorare le proprie prestazioni rispetto all'assenza di apprendimento.

ASSUNZIONE FONDAMENTALE I.I.D

L'apprendimento è possibile solo se il training e l'inferenza sono fatti usando dati dalla stessa distribuzione $x \sim P(X)$. Detto questo, i campioni usati sono indipendenti e identicamente distribuiti (i.i.d), dal training all'inferenza.

2.3.1 Misure di valutazione

Oltre all'accuratezza, che abbiamo visto in precedenza, ci sono molti altri metodi per valutare:

- **Efficienza:**
 - Tempo usato per costruire il modello.
 - Tempo per usare il modello.
- **Robustezza:** gestire il rumore e valori mancanti.
- **Scalibilità:** efficienza in database basati su dischi.
- **Interpretabilità:** comprensibilità e approfondimenti forniti dal modello.
- **Compattezza del modello:** dimensione dell'albero o numero di regole.

2.3.2 Modello di valutazione

Un modello usato è l'**Holdout Set**, dove il dataset D a disposizione viene diviso in due insiemi disgiunti:

- **Insieme per il training**, T_r .
- **Insieme per l'inferenza**, T_e .

NOTA BENE

L'insieme di addestramento non deve essere utilizzato per i test e l'insieme di test non deve essere utilizzato per l'apprendimento. Un insieme di test non visto fornisce una stima imparziale dell'accuratezza.

Questo metodo è solitamente usato quando i dataset D sono molto larghi.

2.4 Iperparametri

Tutti i classificatori presentano parametri aggiuntivi, denominati **iperparametri** e non vengono appresi, bensì selezionati in fase di progettazione prima dell'addestramento.

Diverse configurazioni degli iperparametri possono modificare in modo significativo le prestazioni del modello. Viene usata un'ulteriore suddivisione del dataset D , utilizzata per la messa a punto degli iperparametri e per la selezione del modello migliore, chiamato **insieme di validazione**.

EXAMPLE Train-Validation-Test

- Dato un dataset D , partizioniamo randomicamente i dati in un insieme di allenamento T_r , uno per la validazione V e uno per il test T_e . Di solito con 60/20/20, 80/10/10 o simile.
- I modelli M_i imparano su T_r per ogni scelta dell'iperparametro H_i .
- L'errore di validazione Val_i di ogni modello M_i viene valutato sull'insieme V .
- Vengono selezionati gli iperparametri H_* che presentano il valore più basso di Val_i e il classificatore viene riaddestrato utilizzando tali iperparametri sull'insieme $T_r + V$, ottenendo un modello finale M_* .
- La capacità di generalizzazione viene stimata valutando l'errore o l'accuratezza di M_* sui dati dell'insieme di test T_e .

EXAMPLE k-fold Crossvalidation

Il metodo viene spesso utilizzato quando il dataset non è sufficientemente ampio da consentire di ottenere un test set consistente.

- Partizioniamo randomicamente un dataset D in un insieme di K blocchi $B_1, \dots, B_K$.
- Per ogni fold di cross-validation $k = 1, \dots, K$:
 - Assegna $T_e = B_k$ e $T_r = D - B_k$ (i rimanenti $K - 1$ blocchi).
 - Impara il modello M_k su T_r e testalo su B_k .
- Il processo produce k accurattezze.
- La stima finale delle prestazioni del modello è la media delle K accurattezze.

2.5 Altre Misure di Classificazione

L'accuratezza non è l'unica misura e in alcuni casi non è nemmeno utilizzabile. Per questo vengono usate anche misure:

- **Precisione:** è il numero di esempi positivi classificati correttamente diviso per il numero totale di esempi classificati come positivi.

$$p = \frac{TP}{TP + FP}$$

- **Recall:** è il numero di esempi positivi correttamente classificati diviso per il numero totale di esempi positivi effettivi nell'insieme di test.

$$r = \frac{TP}{TP + FN}$$

- **F1-score:** è la media armonica di precisione e recall. Dà maggior peso al valore più basso.

$$\frac{1}{F1} = \frac{1}{p} + \frac{1}{r}$$

DEFINITION Cross-Entropy

La cross-entropy è una misura della teoria dell'informazione relativa alla codifica dei messaggi utilizzando il numero di bit. Valuta il numero medio di bit per scoprire un dato codificato da una distribuzione Q mentre quello originale era P .

$$\mathbb{E}_P[l_i] = \sum_x P(x) \log_2 \frac{1}{Q(x)} = - \sum_x P(x) \log_2 Q(x)$$

Capitolo 3

Probabilistic Classifier and Linear Classification Models

3.1 Definizioni

DEFINITION Classificazione

Dato un vettore feature $x \in \mathbb{R}^D$ che descrive un oggetto appartenente a una delle classi C dell'insieme $\mathcal{Y}$, predire a quale classe appartenga l'oggetto.

DEFINITION Apprendimento di classificatori

Dato un dataset di coppie di esempi $D = \{(x_i, y_i), i = 1 : N\}$ dove $x \in \mathbb{R}^D$ è un vettore feature e $y_i \in \mathcal{Y}$ è un'etichetta della classe, imparare una funzione $f : \mathbb{R}^D \rightarrow \mathcal{Y}$ che predice accuratamente l'etichetta della classe y per ogni vettore feature x .

Date queste definizioni ed elementi, si possono definire anche due valori:

1. **Tasso di errore di classificazione:** $Err(f, D) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(y_i \neq f(x_i))$
2. **Tasso di accuratezza della classificazione:** $Acc(f, D) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(y_i = f(x_i))$

3.1.1 Formula di Bayes

$$P(Y = c \mid X = x) = \frac{P(X = x \mid Y = c)P(Y = c)}{\sum_{c' \in \mathcal{Y}} P(X = x, Y = c')} = \frac{\phi_c(x)\pi_c}{\sum_{c' \in \mathcal{Y}} \phi_{c'}(x)\pi_{c'}}$$

Dove:

- **Funzione di likelihood:** $P(Y = c) = \pi_{c'}$ definisce la probabilità, che pescando a caso da un dataset D , ottengo una classe c .
- **Funzione di probabilità a priori:** $P(X = x \mid Y = c) = \phi_{c'}(x)$ definisce la probabilità, che nel dataset delle classi c , pescando un x_i a caso, ottengo x .

3.1.2 Maximum Likelihood

Dato un dataset etichettato $D = (x_i, y_i)_{i=1}^N$ con k classi $c_i \mid i = 1 \dots k$ e un campione di test generico $\hat{x}$:

EXAMPLE Approccio

- $P(x \mid c_i)$, trovare una funzione di probabilità p , per ogni classe c_i .
- $\hat{c} = \operatorname{argmax}_c P(\hat{x} \mid c)$ dato un $\hat{x}$ esterno dal dataset D , trovare la classe c_i , per cui la probabilità che $\hat{x}$ appartenga a c_i è maggiore.

3.1.3 Maximum A-Posteriori

Dato un dataset etichettato $D = (x_i, y_i)_{i=1}^N$ con k classi $c_i \mid i = 1 \dots k$ e un campione di test generico $\hat{x}$:

EXAMPLE Approccio

- $P(x \mid c_i)$, trovare una funzione di probabilità p , per ogni classe c_i .
- $\hat{c} = \operatorname{argmax}_c P(\hat{x} \mid c_i)$ dato un $\hat{x}$ esterno dal dataset D , trovare la classe c_i , per cui si ha la probabilità maggiore.

3.2 Classificatore Ottimo di Bayes

Usa la funzione di classificazione:

$$f_B(x) = \arg \max_{c \in \mathcal{Y}} P(Y = c \mid X = x) = \arg \max_{c \in \mathcal{Y}} \phi_c(x) \pi_c$$

E come **errore atteso**:

$$1 - \mathbb{E}_{P(X=x)} \left[\max_{c \in \mathcal{Y}} P(Y = c \mid X = x) \right]$$

Ci sono, però, alcuni problemi, come di casi (x,y) che non sono mai testati (in casi con db piccoli) oppure il fatto che possano esistere classi e feature. La soluzione è calcolare la probabilità $P(x \mid c)$

3.2.1 Stima della densità

Si sono due ricette per stimare la densità:

1. **Maximum Likelihood parametrico**, ossia trovare una funzione di densità, con parametri stimati a:

$$\hat{\Theta} = \arg \max_{\Theta} \prod_{x_i \in D} p(x_i \mid \Theta) = \arg \max_{\Theta} \sum_{x_i \in D} \log(p(x_i \mid \Theta))$$

2. **Stima non parametrica**, partendo da un istogramma con il numero di volte in cui compare x , normalizzare l'istogramma per ottenere le probabilità di x .

3.2.2 Naive Density (Naive DE)

DEFINITION Stima Naive Density

Uno stimatore di densità che assume che tutte le variabili casuali (attributi) di $x \in \mathbb{R}^k$ siano indipendenti.

$$P(x_1, x_2, \dots, x_k \mid c) = \prod_{i=1}^k P(x_i \mid c)$$

Il classificatore **Naive Bayes** approssima il classificatore Bayesiano ottimale utilizzando una forma semplice per la funzione $\phi_c(x)$:

$$\phi_c(x) = p(X = x \mid Y = c) = \prod_{d=1}^k p(X_d = x_d \mid Y = c) = \prod_{d=1}^k \phi_{cd}(x_d)$$

La forma finale è:

$$f_{NB}(x) = \arg \max_{c \in \mathcal{Y}} \pi_c \prod_{d=1}^k \phi_{cd}(x_d)$$

Invece, per stimare c si usa un approccio a priori, senza guardare x , considero solo la probabilità che ho di trovare una classe c :

$$\pi_c = \frac{1}{n} \sum_{i=1}^n [y_i = c]$$

DEFINITION Boundary Conditions

Sono delle rette che dividono il nostro insieme di partenza, in varie porzioni, ognuno per la sua retta. Equivalgono al punto in cui l'equazione è $= 0$.

Nel caso di due sole classi, esce un'equazione quadratica:

$$\sum_{d=1}^D (a_d x_d^2 + b_d x_d) + c = 0$$

Nel caso di classi multiple, l'equazione è quadratica a tratti.

NOTE FINALI

- Il classificatore Naive, è relativamente veloce e facile da implementare.
- Richiede pochi parametri $O(DC)$.
- Si basa sulla condizione di indipendenza (Naive condition), raramente rispettata nel mondo reale e porta a una precisione minore.

3.3 Linear Discriminant Analysis (LDA)

Con LDA, si assume una comune covarianza tra le feature, non si assume quindi l'indipendenza.

La funzione di classificazione è:

$$f_{LDA}(x) = \arg \max_{c \in \mathcal{Y}} \phi_c(x) \pi_c$$

Dove:

$$\phi_c(x) = \mathcal{N}(x; \mu_c, \Sigma) = \frac{1}{|2\pi\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu_c)^T \Sigma^{-1}(x - \mu_c)\right)$$

Come per Naive Bayes, i parametri della LDA vengono appresi utilizzando un approccio maximum likelihood, il che si riduce all'impiego di stime campionarie:

- **Probabilità di classe:** $\pi_c = \frac{1}{n} \sum_{i=1}^n [y_i = c]$
- **Significato di classe:** $\frac{\sum_{i=1}^n [y_i = c] x_i}{\sum_{i=1}^n [y_i = c]}$
- **Covarianza comune:** $\Sigma = \frac{1}{n} \sum_{i=1}^n (x_i - \mu_{y_i})(x_i - \mu_{y_i})^T$

Se le classi hanno la stessa matrice di covarianza, allora le boundary conditions sono lineari in x . Quindi, LDA è un classificatore lineare. Con la covarianza, abbiamo ottenuto 2 bordi distinti, ognuno per ogni classe. Questo ci permette di evitare problemi di sovrapposizione e mascheramento.

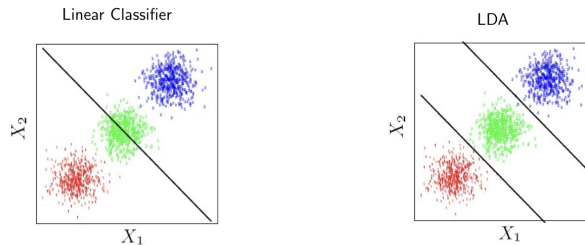


Figura 3.1: Boundary Conditions LDA.

3.3.1 Quadratic Discriminant Analysis

Se durante la scelta della covarianza, si sceglie un modello diverso, è possibile ottenere un modello QDA, un modello non lineare.

Un modello LDA, risulta spesso più robusto, ma l'assunzione di linearità è spesso non rispettata nei casi reali. Un modello QDA, invece è più dinamico, ma è più difficile controllare overfitting e altri problemi.

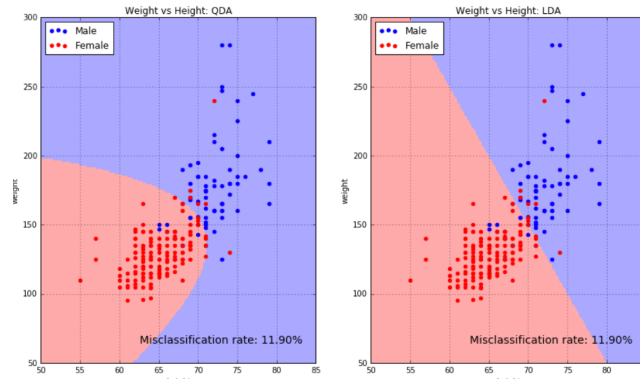


Figura 3.2: Confronto fra LDA e QDA.

NOTE FINALI

- La dipendenza quadratica da D rende l'LDA più lento del Naive Bayes di un fattore D , sia in fase di apprendimento che di classificazione.
- Il modello richiede $O(D^2C)$ parametri. Ciò può comunque rappresentare una buona compressione dei dati quando $D \ll N$.
- In generale, la LDA richiederà più dati rispetto a Naive Bayes, poiché deve stimare i parametri $O(D^2)$ nella matrice di covarianza aggregata Σ .

3.4 Logistic Regression

La logistic regression è un classificatore discriminativo probabilistico A-Posteriori. La sua funzione di classificazione è:

$$f_{LR}(x) = \arg \max_{c \in \mathcal{Y}} P(Y = c | x)$$

Nel caso binario, modella direttamente la probabilità a posteriori come:

$$P(Y = 0 | x) = \frac{e^{w^T x + b}}{1 + e^{w^T x + b}}$$

$$P(Y = 1 | x) = \frac{1}{1 + e^{w^T x + b}}$$

Calcolando le boundary conditions, si dimostra che il che è lineare. C'è anche il caso con K classi:

$$P(Y = c | x) = \frac{\exp(w_c^T x + b_c)}{1 + \sum_{l=1}^{K-1} \exp(w_l^T x + b_l)}$$

$$P(Y = K | x) = \frac{1}{1 + \sum_{l=1}^{K-1} \exp(w_l^T x + b_l)}$$

3.4.1 Ottimizzazione dei Parametri

I parametri del modello di regressione logistica $\theta = \{(w_c, b_c), c \in (y)\}$ vengono selezionati per ottimizzare la log likelihood condizionata delle etichette dato un set di dati $D = \{(x_i, y_i), i = 1 : n\}$:

$$\theta_* = \arg \max_{\theta} \mathcal{L}(\theta | D) = \arg \max_{\theta} \sum_{i=1}^n \log P(Y = y_i | X = x_i)$$

Tuttavia, la funzione $\mathcal{L}(\theta | D)$ non può essere massimizzata analiticamente. L'apprendimento dei parametri del modello richiede metodi di ottimizzazione numerica.

3.4.2 Discesa del Gradiente

DEFINITION Discesa del Gradiente

La discesa del gradiente è un algoritmo di ottimizzazione iterativo per trovare il minimo di una funzione. Eseguire un passo proporzionale all'opposto del gradiente della funzione nel punto corrente.

Se consideriamo una funzione $f(\theta)$, l'**aggiornamento della discesa** del gradiente, per ogni parametro θ_j , può essere espresso come:

$$\theta_j = \theta_j - \alpha \frac{d}{d\theta_j} f(\theta)$$

La dimensione del passo è controllata dal **tasso di apprendimento** α . Scegliere il tasso di apprendimento α corretto è fondamentale per procedere correttamente verso il minimo:

- Un passo troppo piccolo potrebbe portare a una convergenza estremamente lenta.
- Se il passo è troppo ampio, l'ottimizzatore potrebbe oltrepassare il minimo o addirittura divergere.

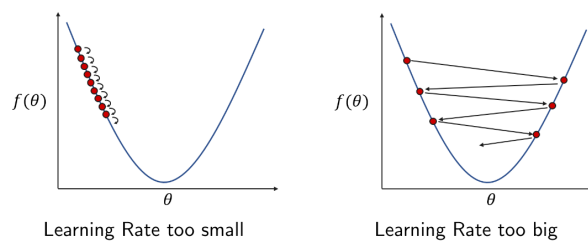


Figura 3.3: Scelta tasso di apprendimento.

Esistono diversi tipi di discesa del gradiente:

- **Batch gradient descent:** ottimizzazione su l'intero batch di dati.
- **Mini batch stochastic gradient descent:** ad ogni step considero solo un piccolo insieme di dati, presi dal training set.
- **Stochastic gradient descent:** caso in cui per ogni step considero solo 1 esempio.

CONVESSITÀ

Se la funzione è convessa, la discesa del gradiente converge al **minimo globale**. Nel caso di funzioni non convexe, può convergere a **minimi locali**.

NOTE FINALI

- In fase di classificazione, la LR è più veloce di NB e LDA. L'apprendimento della LR richiede un'ottimizzazione numerica, che risulta più lenta rispetto a NB.
- Richiede gli stessi parametri degli altri modelli (o meno se $C \ll D$).
- Ha più accuratezza in casi lineari con tante classi.

Capitolo 4

Support Vector Machine

4.1 Classificazione Binaria

Ci troviamo di fronte a un problema di classificazione binaria, dove dobbiamo distinguere i puntini blu da quelli rossi.

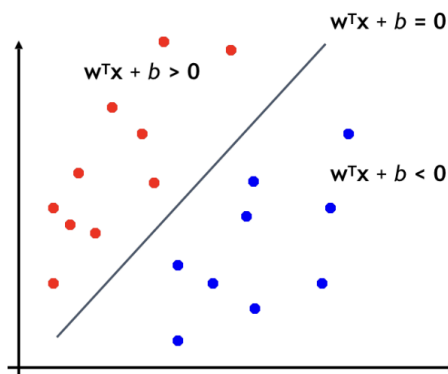


Figura 4.1: Esempio di problema di classificazione binaria.

Bisogna trovare la retta $w^T x + b$ che separi le due classi, ma esistono infinite rette. Va, quindi, trovata quella **ottima**.

Per fare questo, possiamo definire dei **vettori di supporto** e calcolare la loro distanza come:

$$r_i = \frac{w^T x_i + b}{\|w\|}$$

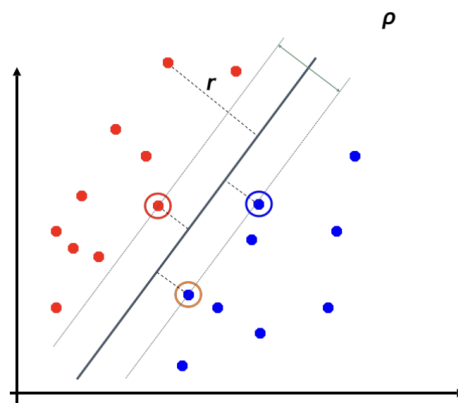


Figura 4.2: Utilizzo vettori di supporto.

L'obiettivo è massimizzare questa quantità per tutti gli x_i . Puntiamo al boundary che offre la maggiore sicurezza.

EXAMPLE Calcolo dei margini

Dato un dataset $D = (x_i, y_i)_{i \in [1, \dots, N]}$ e $y_i \in \{-1, 1\}$ con ρ come margine, vogliamo che:

$$y_i(w^T x_i + b) \geq \rho/2 \quad \forall x_i \in D$$

Per il vettore di supporto x_s l'equazione diventa $w^T x_s + b = \rho/2$, ottenendo quindi questa espressione per il margine:

$$r_s = \frac{w^T x_s + b}{\|w\|} = \frac{\rho}{2\|w\|} = \frac{1}{\|\hat{w}\|}$$

$$\rho = 2r_s = \frac{2}{\|w\|}$$

4.1.1 Problema quadratico SVM

La formula di **massimizzazione** del margine:

$$\arg \max_{w, b} \frac{2}{\|w\|} \quad s.t.$$

$$y_i(w^T x_i + b) \geq 1 \quad \forall (x_i, y_i) \in D$$

Equivalente a quella di **minimizzazione** dei pesi:

$$\arg \min_{w, b} \|w\|^2 \quad s.t.$$

$$y_i(w^T x_i + b) \geq 1 \quad \forall (x_i, y_i) \in D$$

4.1.2 Ottimizzazione Vincolata

In alcuni casi, potrebbe essere necessario ottimizzare problemi condizionati:

$$\min f(x)$$

$$\text{subject to } h_i(x) \geq 0 \quad \text{for } i \in 1, \dots, N$$

DEFINITION Condizioni di Lagrange

La funzione Lagrangiana è:

$$L(x, \alpha) = f(x) - \sum_i^N \alpha_i h_i(x) \quad \text{con } \alpha_i \geq 0$$

Dove:

- Se per x_i , $h(x_i) > 0$ allora $\alpha = 0$ il vincolo si dice **inattivo** e la soluzione è $x^* \mid \nabla f(x) = 0$
- Se per x_i , $h(x_i) = 0$ allora $\alpha > 0$ il vincolo si dice **attivo** e la soluzione è $x^* \mid \nabla f(x) - \alpha^T \nabla h(x) = 0$

Mettendo tutto insieme, troviamo le condizioni di **Karush-Kuhn-Tucker (KKT)**, che costituiscono condizioni necessarie per l'esistenza di una soluzione unica x_* :

$$\nabla f(x^*) - \alpha^T \nabla h(x^*) = 0$$

$$h(x^*) \geq 0$$

$$\alpha_i \geq 0$$

$$\alpha^T h(x) = 0$$

4.1.3 Classificatore SVM

La funzione di classificazione SVM lineare nella forma **primale**, inglobando b nei pesi w con una dimensione in più, è:

$$f(x) = w^T x$$

Invece, nella forma **duale** è:

$$f(x) = \sum_{i=1}^N a_i y_i x_i^T x$$

MARGINI NON SEPARABILI

Se i pattern dei dati non sono separabili mediante un iperpiano lineare, un insieme di **variabili di slack** $\epsilon = \epsilon_1, \epsilon_2, \dots, \epsilon_N$ sono introdotte con $\epsilon_i \geq 0$ facendo diventare i vincoli dell'SVM:

$$y_i(x_i w + b) \geq 1 - \epsilon_i$$

Inoltre, il problema di ottimizzazione diventa:

$$\min \frac{1}{2} \|w\|^2 + C \sum_i \epsilon_i$$

$$\text{subject to } y_i(w^T x_i + b) \geq 1 - \epsilon_i \quad \text{con } \epsilon_i \geq 0$$

4.1.4 SVM non Lineare

Cosa succede se le classi non sono separabili linearmente?

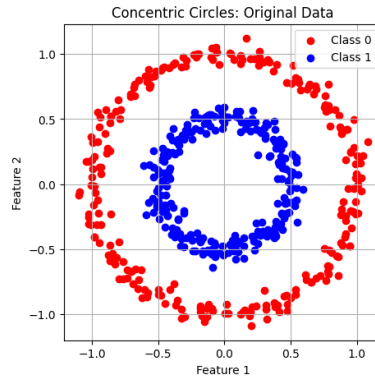


Figura 4.3: Grafico di un SVM non lineare.

La soluzione migliore, è trasformare lo spazio di classificazione in uno in cui il confine è lineare.

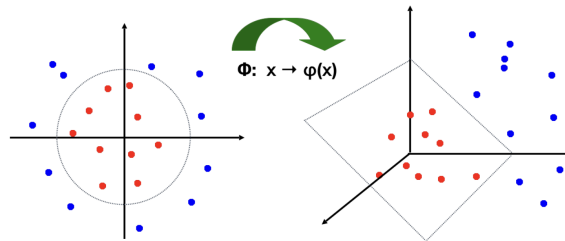


Figura 4.4: Grafico sul cambio di spazio di classificazione.

Potremmo utilizzare una mappatura basata su una funzione non lineare: $\phi : x \rightarrow \phi(x)$

DEFINITION Funzione di Kernel

Una funzione kernel mappa implicitamente i dati in uno spazio ad alta dimensionalità senza la necessità di calcolare esplicitamente ogni $\phi(x)$.

EXAMPLE Definizione funzione kernel

Dato $f(x) = \sum_{i=1}^N a_i y_i (x_i^T x)$:

1. **Mappatura non lineare:** $\phi : x \rightarrow \phi(x)$.
2. **Definire la kernel function:** $\phi(x_i)^T \phi(x_j) = K(x_i, x_j)$.
3. **Sostituzione:** $f(x) = \sum_{i=1}^N a_i y_i K(x_i, x_j)$.

Ci sono, inoltre, vari tipi di kernel:

- **Lineare:** $K(x, x_i) = x^T x_i$, utilizzato dal classificatore SVM classico.
- **Polinomiale:** $K(x, x_i) = (1 + x^T x_i)^p$, espande in maniera polinomiale le dimensioni come $\binom{d+p}{d}$.
- **RBF (Gaussiano):** $K(x, x_i) = \exp(-\frac{\|x-x_i\|^2}{2\sigma^2})$, espande in una dimensione infinita, come $e^x = \sum_{n=1}^{\infty} \frac{x^n}{n!}$.